



# Centro Deportivo

Integrantes:

Bassi Servant, Tomas

Benedetti, Matias

Universidad Tecnológica Nacional

Gestión de desarrollo de software

Profesor:

Mango, Eduardo

Fecha:

17/11/2025

# Índice

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>Índice.....</b>                                                                             | <b>2</b>  |
| <b>Introducción.....</b>                                                                       | <b>5</b>  |
| Misión.....                                                                                    | 5         |
| Alcance.....                                                                                   | 5         |
| Descripción general.....                                                                       | 5         |
| <b>Requisitos funcionales.....</b>                                                             | <b>6</b>  |
| Listado.....                                                                                   | 6         |
| Especificación.....                                                                            | 7         |
| RF-01: El usuario anónimo puede consultar el listado de todas las actividades disponibles..... | 7         |
| RF-03: El usuario anónimo puede registrarse como socio.....                                    | 8         |
| RF-10: Socio puede inscribirse a una actividad.....                                            | 8         |
| RF-13: El socio puede editar su información personal.....                                      | 9         |
| RF-21: El instructor puede inscribir un socio en una actividad que dicta.....                  | 9         |
| RF-26: El administrador puede eliminar socios e instructores.....                              | 10        |
| Tabla de alcance final.....                                                                    | 11        |
| <b>Medición funcional.....</b>                                                                 | <b>11</b> |
| Tabla de Conteo detallada.....                                                                 | 11        |
| Cálculo de Puntos de Función No Ajustados (UFP).....                                           | 13        |
| Tabla de Asignación del Grado de Influencia (GI).....                                          | 14        |
| ESTIMACIÓN DE ESFUERZO, TIEMPO Y COSTO.....                                                    | 15        |
| Técnica Obligatoria 1: COCOMO.....                                                             | 15        |
| ESFUERZO.....                                                                                  | 16        |
| TIEMPO.....                                                                                    | 16        |
| PERSONAS PROMEDIO.....                                                                         | 16        |
| COSTO TOTAL.....                                                                               | 16        |
| Explicación de la Elección del Tipo de Proyecto.....                                           | 16        |
| Técnica Obligatoria 2: PLANNING POKER.....                                                     | 17        |
| Resultados Clave.....                                                                          | 23        |
| Similitudes entre COCOMO y Planning Poker.....                                                 | 23        |
| Diferencias Clave.....                                                                         | 24        |
| Análisis de la Diferencia en el Tiempo de Desarrollo.....                                      | 24        |
| COCOMO (8.3 meses): Esfuerzo de Proceso Total.....                                             | 24        |
| Planning Poker (5.68 meses): Esfuerzo de Desarrollo Puro.....                                  | 24        |
| Conclusión.....                                                                                | 25        |
| <b>Arquitectura del sistema - Modelo C4.....</b>                                               | <b>25</b> |
| Nivel 1: Diagrama de contexto.....                                                             | 25        |
| Descripción.....                                                                               | 25        |
| Tecnologías.....                                                                               | 26        |
| Responsabilidades.....                                                                         | 26        |
| Nivel 2: Diagrama de contenedores.....                                                         | 27        |
| Descripción.....                                                                               | 27        |



|                                                                                                       |           |
|-------------------------------------------------------------------------------------------------------|-----------|
| Responsabilidades.....                                                                                | 27        |
| Nivel 3: Diagrama de componentes.....                                                                 | 28        |
| Frontend.....                                                                                         | 28        |
| Responsabilidades.....                                                                                | 28        |
| Backend.....                                                                                          | 29        |
| Responsabilidades.....                                                                                | 29        |
| <b>Gestión de riesgos.....</b>                                                                        | <b>30</b> |
| <b>Decisiones claves.....</b>                                                                         | <b>31</b> |
| ADR 001: Integración de Chatbot conversacional con IA mediante Google Gemini.....                     | 31        |
| Estado.....                                                                                           | 31        |
| Contexto.....                                                                                         | 31        |
| Alternativas consideradas.....                                                                        | 31        |
| Decisión.....                                                                                         | 32        |
| Consecuencias.....                                                                                    | 32        |
| Impactos positivos.....                                                                               | 32        |
| Impactos negativos.....                                                                               | 33        |
| ADR 002: Selección de arquitectura de persistencia para el módulo de rutinas de<br>entrenamiento..... | 33        |
| Estado.....                                                                                           | 33        |
| Contexto.....                                                                                         | 33        |
| Alternativas consideradas.....                                                                        | 33        |
| Decisión.....                                                                                         | 34        |
| Consecuencias.....                                                                                    | 34        |
| Impactos positivos.....                                                                               | 34        |
| Impactos negativos.....                                                                               | 35        |
| <b>SCRUM.....</b>                                                                                     | <b>35</b> |
| Definición de roles.....                                                                              | 35        |
| Product Backlog.....                                                                                  | 35        |
| EPIC/Gestión de acceso.....                                                                           | 36        |
| EPIC/Catálogo de actividades.....                                                                     | 36        |
| EPIC/Gestión de socios.....                                                                           | 36        |
| EPIC/Funcionalidades para administradores.....                                                        | 36        |
| EPIC/Funcionalidades para instructores.....                                                           | 36        |
| EPIC/Gestión de rutinas.....                                                                          | 36        |
| EPIC/Asistente virtual.....                                                                           | 37        |
| <b>Artefactos.....</b>                                                                                | <b>37</b> |
| Sprint planning.....                                                                                  | 37        |
| Sprint 1.....                                                                                         | 37        |
| Sprint 2.....                                                                                         | 37        |
| Sprint 3.....                                                                                         | 37        |
| Sprint 4.....                                                                                         | 37        |
| Sprint 5.....                                                                                         | 38        |
| Sprint review.....                                                                                    | 38        |
| Sprint retrospective.....                                                                             | 38        |



|                                                |           |
|------------------------------------------------|-----------|
| Sprint 1.....                                  | 38        |
| ¿Qué funcionó bien?.....                       | 38        |
| ¿Qué se puede mejorar?.....                    | 38        |
| Acciones de mejora para el próximo sprint..... | 38        |
| Sprint 2.....                                  | 39        |
| ¿Qué funcionó bien?.....                       | 39        |
| ¿Qué se puede mejorar?.....                    | 39        |
| Acciones de mejora para el próximo sprint..... | 39        |
| Sprint 3.....                                  | 39        |
| ¿Qué funcionó bien?.....                       | 39        |
| ¿Qué se puede mejorar?.....                    | 39        |
| Acciones de mejora para el próximo sprint..... | 39        |
| Sprint 4.....                                  | 39        |
| ¿Qué funcionó bien?.....                       | 39        |
| ¿Qué se puede mejorar?.....                    | 39        |
| Acciones de mejora para el próximo sprint..... | 39        |
| Sprint 5.....                                  | 40        |
| ¿Qué funcionó bien?.....                       | 40        |
| ¿Qué se puede mejorar?.....                    | 40        |
| Acciones de mejora para el próximo sprint..... | 40        |
| <b>Gestión de cambios.....</b>                 | <b>40</b> |
| <b>Cierre.....</b>                             | <b>41</b> |
| Comparación estimación vs real.....            | 41        |
| Tabla Comparativa Final.....                   | 41        |
| Análisis de desviaciones.....                  | 42        |
| Esfuerzo, Tiempo y Costo (aprox. -67%).....    | 42        |
| Causa Raíz de la Desviación.....               | 42        |
| ¿Se podría haber previsto?.....                | 42        |
| ¿Qué acciones se tomaron al detectarla?.....   | 42        |
| ¿Cómo impactó en el resultado final?.....      | 42        |

# Introducción

## Misión

Nuestra misión es centralizar y digitalizar la totalidad de la experiencia del centro deportivo. Buscamos trascender la simple gestión administrativa de clases e inscripciones para convertir la plataforma en un compañero de fitness integral para el socio. El objetivo es empoderar a los socios con herramientas para visualizar y alcanzar sus metas de entrenamiento, al mismo tiempo que se dota a instructores y administradores de un potente software de gestión para optimizar la asignación de recursos y el seguimiento de miembros. Adicionalmente, se integra un asistente virtual conversacional impulsado por IA para brindar soporte instantáneo y personalizado a los usuarios.

## Alcance

- Módulo de gestión de usuarios (Auth/Admin): Un sistema de autenticación basado en JWT y autorización por roles (Socio, Instructor, Admin) y permisos granulares (PermissionLevel para Admin). Incluye el CRUD completo de usuarios y la autogestión de perfiles.
- Módulo de gestión de actividades: La gestión de inscripciones y bajas de socios a las actividades deportivas, así como la consulta y búsqueda de las mismas.
- Módulo de fitness y seguimiento: Un sistema completo para la creación de rutinas, asignación de estas a los socios y, de forma crucial, el registro y visualización gráfica del progreso personal.
- Módulo de asistencia virtual (Chatbot): Un chatbot conversacional (ChatBum) que utiliza Google Gemini AI para responder consultas en lenguaje natural sobre actividades, horarios, instructores y disponibilidad. Proporciona información personalizada a usuarios sobre sus inscripciones y estado.

## Descripción general

Este proyecto es una solución web integral que moderniza la administración de un centro deportivo. Se compone de un backend (API REST) desarrollado en Java 21 con Spring Boot, que maneja toda la lógica de negocio, la seguridad con JWT y la persistencia de datos en una base de datos MySQL. La arquitectura del backend sigue un diseño en capas (Controller, Service, Repository), con manejo centralizado de excepciones y validación de entradas.

El frontend es una SPA Angular que consume la API, ofreciendo una experiencia de usuario fluida y adaptada a cada rol:

1. Público General: Puede explorar las actividades y registrarse o iniciar sesión. También puede consultar al asistente virtual sobre información general de las actividades disponibles.
2. Socios: Acceden a su perfil, gestionan sus inscripciones a clases y, como funcionalidad central, pueden crear sus propias rutinas, ver las asignadas, cargar su progreso y visualizar su evolución en gráficos. El chatbot les proporciona asistencia

personalizada sobre sus inscripciones activas y responde dudas sobre las actividades del centro.

3. Instructores: Además de gestionar sus clases e inscripciones, pueden registrar nuevos socios y actuar como entrenadores personales asignando y editando rutinas. Pueden consultar al chatbot información sobre las actividades que dictan y los socios inscriptos.
4. Administradores: Tienen control total sobre la plataforma, gestionando todas las cuentas de usuario, las inscripciones y también pueden asignar rutinas.

El sistema integra un widget de chat flotante disponible en todas las páginas, que mantiene la conversación y accede a información en tiempo real del backend para proporcionar respuestas actualizadas y contextualizadas mediante inteligencia artificial generativa.

## Requisitos funcionales

### Listado

- RF-01. El usuario anónimo puede consultar el listado de todas las actividades disponibles.
- RF-02. El usuario anónimo puede visualizar el detalle de una actividad seleccionada, incluyendo horario, instructor y precio.
- RF-03. El usuario anónimo puede registrarse como socio.
- RF-04. El usuario anónimo puede iniciar sesión.
- RF-05. El socio puede consultar todas las actividades disponibles.
- RF-06. El socio puede ver el detalle de una actividad (nombre, instructor, horario, precio).
- RF-07. El socio puede buscar actividades por nombre.
- RF-08. El socio puede buscar actividades por rango horario.
- RF-09. El socio puede ver el detalle del instructor que dicta una actividad.
- RF-10. El socio puede inscribirse a una actividad.
- RF-11. El socio puede ver el listado de actividades en las que está inscripto.
- RF-12. El socio puede darse de baja de una actividad.
- RF-13. El socio puede editar su información personal.
- RF-14. El socio puede eliminar su cuenta.
- RF-15. El instructor puede registrar nuevos socios.
- RF-16. El instructor puede listar a todos los socios.
- RF-17. El instructor puede ver el detalle de un socio.
- RF-18. El instructor puede ver el listado de todas las actividades del sistema.
- RF-19. El instructor puede ver el listado de actividades que dicta.
- RF-20. El instructor puede ver el detalle de una actividad.
- RF-21. El instructor puede inscribir un socio en una actividad que dicta.
- RF-22. El instructor puede dar de baja a un socio de una actividad que dicta.
- RF-23. El administrador puede crear socios, instructores y otros administradores.
- RF-24. El administrador puede listar todos los usuarios (socios, instructores, administradores).
- RF-25. El administrador puede ver el detalle de cualquier usuario y actividad.
- RF-26. El administrador puede eliminar socios e instructores.
- RF-27. El administrador puede inscribir a un socio en cualquier actividad.

- RF-28. El administrador puede dar de baja a un socio de cualquier actividad.
- RF-29. Todos los usuarios pueden crear rutinas.
- RF-30. El instructor puede asignar rutinas.
- RF-31. El socio puede ver su rutina.
- RF-32. El socio puede cargar su progreso.
- RF-33. El socio puede ver gráficos con su progreso por ejercicio por rutina.
- RF-34. El socio puede eliminar su rutina.
- RF-35. El instructor puede editar rutinas.
- RF-36. El socio puede ver su rutina.
- RF-37. El usuario anónimo puede interactuar con el chatbot para consultar información general sobre actividades.
- RF-38. El socio puede usar el chatbot para consultar información personalizada sobre sus inscripciones.
- RF-39. El chatbot puede proporcionar información sobre horarios, instructores y disponibilidad de cupos de las actividades.
- RF-40. El chatbot puede responder preguntas en lenguaje natural sobre las actividades del centro deportivo.

## Especificación

**RF-01: El usuario anónimo puede consultar el listado de todas las actividades disponibles.**

Descripción: Permite a cualquier tipo de usuario (autenticado o no) consultar la lista completa de actividades deportivas disponibles en el centro.

Contexto: Requisito informativo al que se accede libremente.

Objetivo: Mostrar la oferta de clases de forma accesible para que los usuarios puedan explorarlas e inscribirse.

Entradas: Parámetros opcionales de paginación: page (número de página), size (elementos por página) y filtros de búsqueda.

Salidas: Lista paginada de actividades. Cada elemento incluye: Nombre, Capacidad Máxima y Nombre del Instructor.

Proceso: 1. El sistema recibe la solicitud de listado (con o sin parámetros de paginación). 2. El SportActivityService consulta el ISportActivityRepository. 3. Se obtienen y procesan los datos de todas las actividades. 4. Se estructura la respuesta con la información resumida y la paginación aplicada. 5. Se devuelve el listado al usuario.

Reglas de negocio: - El acceso es público (no requiere autenticación). - La respuesta debe ser paginada y configurable.

Prioridad: ALTA.

### RF-03: El usuario anónimo puede registrarse como socio.

Descripción: Permite que un usuario no autenticado se registre en el sistema para convertirse en un nuevo socio del centro deportivo.

Contexto: Requisito inicial (del usuario anónimo) para acceder al sistema. Su éxito lleva a la posibilidad de **RF-04** (Iniciar Sesión).

Objetivo: Capturar los datos personales y credenciales de un nuevo usuario, estableciéndolo como socio en el sistema con un estado inicial **INACTIVE**.

Entradas: Nombre, Apellido, DNI, Fecha de Nacimiento, Teléfono, Email, Nombre de Usuario, Contraseña.

Salidas: Mensaje de confirmación o error (si DNI, Email o Nombre de Usuario ya existen, o por fallos de validación).

Proceso: 1. Usuario no autenticado accede al formulario de registro. 2. Ingresa y envía sus datos. 3. El sistema valida el formato de todos los campos. 4. El sistema verifica la unicidad de DNI, Email y Nombre de Usuario. 5. La contraseña se encripta. 6. El nuevo usuario se persiste con el rol de SOCIO y **Status INACTIVE**. 7. Se notifica el éxito del registro.

Reglas de negocio: - El DNI, Email y Nombre de Usuario deben ser únicos en el sistema.  
- El estado inicial del socio al registrarse es **INACTIVE**.

Prioridad: ALTA.

### RF-10: Socio puede inscribirse a una actividad

Descripción: El socio autenticado puede inscribirse de forma autónoma en una actividad deportiva disponible en el catálogo, asegurando su lugar en la clase y gestionando su estado en el sistema.

Contexto: Requiere que el socio esté autenticado (**RF-04**). El socio puede ver las actividades disponibles a través de **RF-05 / RF-06**.

Objetivo: Permitir la autogestión de clases por parte del socio.

Entradas: ID de la actividad (en la URL), Token JWT del socio (en cabecera).

Salidas: Mensaje de confirmación de registro (Enrollment creado exitosamente); Mensaje de error (MaxCapacityException, MemberAlreadyEnrolledException, SportActivityNotFoundException).

Proceso: 1. El socio autenticado selecciona una actividad y solicita la inscripción. 2. El sistema verifica la existencia de la actividad (SportActivityNotFoundException). 3. El sistema valida que la actividad no haya alcanzado el cupo máximo (maxMembers). 4. El sistema verifica que el socio no esté ya inscrito en esa actividad. 5. Si es válido, se crea un registro de Enrollment (inscripción) con fecha de inicio actual y fecha de fin (30 días



después). 6. El sistema verifica el Status actual del socio. Si es INACTIVE, lo actualiza a ACTIVE. 7. Se notifica la inscripción exitosa.

Reglas de negocio: - El Status del socio cambia a ACTIVE si estaba INACTIVE tras la primera inscripción exitosa. - No se permite la inscripción si el cupo (maxMembers) está lleno. - No se permite la inscripción si el socio ya tiene un registro activo en esa actividad. - La inscripción siempre genera un registro con fecha de fin (ej. 30 días) para indicar la duración de la membresía en esa clase.

Prioridad: ALTA.

### RF-13: El socio puede editar su información personal.

Descripción: Permite al socio autenticado actualizar los datos de su perfil, como nombre, apellido, teléfono, email y fecha de nacimiento, para mantener su información al día en el sistema.

Contexto: Requiere que el socio esté autenticado (**RF-04**). Los datos a editar provienen del **RF-03** (Registro de Socios) inicial o de **RF-23** (Creación por Admin).

Objetivo: Permitir la autogestión de la información personal del socio, garantizando la actualidad y precisión de los datos.

Entradas: Nombre, Apellido, Teléfono, Email, Fecha de Nacimiento (parcial o totalmente); Token JWT del socio (en cabecera).

Salidas: Datos del socio actualizados; Mensaje de error (por email duplicado o formato inválido).

Proceso: 1. El socio autenticado envía los nuevos datos a actualizar. 2. El sistema verifica el Token JWT y obtiene el ID del usuario. 3. El sistema valida los formatos de los campos recibidos (ej. email, fecha de nacimiento). 4. Si el email es enviado, el sistema verifica que no esté ya asociado a otro usuario. 5. Si es válido, la capa MemberService actualiza solo los campos proporcionados en la base de datos. 6. Se retorna la información actualizada del socio.

Reglas de negocio: - El email, si es modificado, debe seguir siendo único en el sistema. - Se deben aplicar las mismas validaciones de formato que en el registro. - Los campos no incluidos en la solicitud **no deben** modificarse (actualización parcial).

Prioridad: MEDIA.

### RF-21: El instructor puede inscribir un socio en una actividad que dicta.

Descripción: Permite que un instructor registre manualmente a un socio en una actividad que él mismo imparte.

Contexto: Requisito que requiere **Autenticación (RF-03)** y es complementario al proceso

de inscripción por parte del propio socio.

Objetivo: Facilitar al instructor la gestión directa de las listas de su propia clase.

Entradas: ID del Socio (memberId), ID de la Actividad (activityId).

Salidas: Datos de la inscripción exitosa; UnauthorizedException, MaxCapacityException, MemberAlreadyEnrolledException, MemberNotFoundException, SportActivityNotFoundException.

Proceso: 1. El instructor autenticado solicita inscribir un socio en una actividad. 2. El sistema verifica si el instructor autenticado es el responsable de la actividad (activityId). 3. Se verifica que la actividad no haya excedido maxMembers. 4. Se verifica que el socio no esté ya inscrito. 5. Si todas las validaciones pasan, se crea la inscripción y se cambia el Status del socio a ACTIVE si era INACTIVE.

Reglas de negocio: - El instructor solo puede inscribir socios en sus propias actividades. - No se permite la inscripción si se supera maxMembers. - Si el socio estaba INACTIVE, su estado cambia a ACTIVE con la primera inscripción.

Prioridad: ALTA.

## RF-26: El administrador puede eliminar socios e instructores.

Descripción: Permite a un administrador con permisos adecuados eliminar a otros usuarios (Socios, Instructores o, según el rol, otros Administradores) del sistema.

Contexto: Requisito exclusivo de gestión y mantenimiento, que depende directamente de **RF-04** (Iniciar Sesión) y se complementa con **RF-24** (Listar todos los usuarios) para la identificación del usuario a eliminar. Su éxito revierte el efecto de **RF-03** y **RF-23**.

Objetivo: Mantener la base de usuarios limpia, gestionar el acceso y revocar cuentas de forma segura.

Entradas: ID o Nombre de usuario a eliminar.

Salidas: Mensaje de éxito de la eliminación; UserNotFoundException; UnauthorizedException

Proceso: 1. El administrador autenticado solicita la eliminación de un usuario. 2. El sistema verifica los permisos del administrador (según su rol: USER\_MANAGER o SUPER\_ADMIN). 3. El sistema busca el usuario a eliminar. 4. Si el usuario existe y los permisos son válidos, se procede a la eliminación a través de AdminService. 5. Si los permisos son insuficientes o el usuario no existe, se lanza la excepción correspondiente.

Reglas de negocio: - Un USER\_MANAGER solo puede eliminar Socios e Instructores. - Un SUPER\_ADMIN no puede eliminarse a sí mismo ni a otro SUPER\_ADMIN. - La eliminación se realiza por ID o Nombre de Usuario.

Prioridad: MEDIA.



## Tabla de alcance final

| Código | Nombre                                                                              | Prioridad | Implementado |
|--------|-------------------------------------------------------------------------------------|-----------|--------------|
| RF-01  | El usuario anónimo puede consultar el listado de todas las actividades disponibles. | Alta      | SI           |
| RF-03  | El usuario anónimo puede registrarse como socio.                                    | Alta      | SI           |
| RF-10  | Socio puede inscribirse a una actividad                                             | Alta      | Si           |
| RF-26  | El administrador puede eliminar socios e instructores.                              | Media     | Si           |
| RF-21  | El instructor puede inscribir un socio en una actividad que dicta.                  | Alta      | Si           |
| RF-13  | El socio puede editar su información personal.                                      | Media     | Si           |

## Medición funcional

### Tabla de Conteo detallada

| Código | Descripción Breve                       | SP | Tipo de Función | Complejidad Estimada | Puntos Asignados |
|--------|-----------------------------------------|----|-----------------|----------------------|------------------|
| RF-01  | Consultar listado actividades (Anónimo) | 3  | EO              | Baja                 | 4                |
| RF-02  | Ver detalle actividad (Anónimo)         | 1  | EQ              | Baja                 | 3                |
| RF-03  | Registrar Socio                         | 5  | EI              | Media                | 4                |
| RF-04  | Iniciar Sesión                          | 5  | EI              | Media                | 4                |
| RF-05  | Consultar listado actividades (Socio)   | 3  | EO              | Baja                 | 4                |



|       |                                                       |   |    |       |   |
|-------|-------------------------------------------------------|---|----|-------|---|
| RF-06 | Ver detalle actividad (Socio)                         | 1 | EQ | Baja  | 3 |
| RF-07 | Buscar actividades por nombre                         | 5 | EQ | Media | 4 |
| RF-08 | Buscar actividades por rango horario                  | 5 | EQ | Media | 4 |
| RF-09 | Ver detalle del instructor                            | 3 | EQ | Baja  | 3 |
| RF-10 | Socio puede inscribirse a actividad                   | 3 | EI | Baja  | 3 |
| RF-11 | Ver listado actividades inscriptas                    | 3 | EO | Baja  | 4 |
| RF-12 | Socio puede darse de baja de actividad                | 2 | EI | Baja  | 3 |
| RF-13 | Socio puede editar su info personal                   | 3 | EI | Baja  | 3 |
| RF-14 | Socio puede eliminar su cuenta                        | 2 | EI | Baja  | 3 |
| RF-15 | Instructor puede registrar nuevos socios              | 2 | EI | Baja  | 3 |
| RF-16 | Instructor puede listar a todos los socios            | 2 | EO | Baja  | 4 |
| RF-17 | Ver detalle de un socio                               | 3 | EQ | Baja  | 3 |
| RF-18 | Instructor puede ver listado de todas las actividades | 3 | EO | Baja  | 4 |
| RF-19 | Instructor puede ver listado de actividades que dicta | 3 | EO | Baja  | 4 |
| RF-20 | Ver detalle actividad (Instructor)                    | 1 | EQ | Baja  | 3 |
| RF-21 | Instructor puede inscribir socio                      | 3 | EI | Baja  | 3 |
| RF-22 | Instructor puede dar de baja a socio                  | 2 | EI | Baja  | 3 |
| RF-23 | Admin puede crear usuarios                            | 5 | EI | Media | 4 |
| RF-24 | Admin puede listar todos los usuarios                 | 3 | EO | Baja  | 4 |



|       |                                                        |   |    |       |   |
|-------|--------------------------------------------------------|---|----|-------|---|
| RF-25 | Admin puede ver detalle de cualquier usuario/actividad | 2 | EQ | Baja  | 3 |
| RF-26 | Admin puede eliminar socios/instructores               | 3 | EI | Baja  | 3 |
| RF-27 | Admin puede inscribir a un socio                       | 2 | EI | Baja  | 3 |
| RF-28 | Admin puede dar de baja a un socio                     | 2 | EI | Baja  | 3 |
| RF-29 | Todos pueden crear rutinas                             | 5 | EI | Media | 4 |
| RF-30 | Instructor puede asignar rutinas                       | 2 | EI | Baja  | 3 |
| RF-31 | Socio puede ver su rutina                              | 2 | EQ | Baja  | 3 |
| RF-32 | Socio puede cargar su progreso                         | 3 | EI | Baja  | 3 |
| RF-33 | Socio puede ver gráficos con su progreso               | 3 | EO | Baja  | 4 |
| RF-34 | Socio puede eliminar su rutina                         | 3 | EI | Baja  | 3 |
| RF-35 | Instructor puede editar rutinas                        | 5 | EI | Media | 4 |
| RF-36 | Socio puede ver su rutina                              | 2 | EQ | Baja  | 3 |
| RF-37 | Chabot consulta información general                    | 5 | EQ | Media | 4 |
| RF-38 | Chabot consulta información personalizada              | 5 | EQ | Media | 4 |
| RF-39 | Chabot consulta horario, instructor, cupos             | 5 | EQ | Media | 4 |
| RF-40 | Chabot responde preguntas lenguaje natural             | 5 | EQ | Media | 4 |

## Cálculo de Puntos de Función No Ajustados (UFP)

$$UFP = \sum PF = 137$$

Tabla de Asignación del Grado de Influencia (GI)

| # | Característica General del Sistema  | Influencia (0-5) | Justificación                                                                                                                       |
|---|-------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Comunicación de Datos               | 4                | Alta dependencia de la comunicación entre frontend, API y base de datos, además de las interacciones con el chatbot (RF-37, RF-38). |
| 2 | Procesamiento de Datos Distribuidos | 0                | Asumimos que no hay procesamiento distribuido o microservicios complejos.                                                           |
| 3 | Performance                         | 4                | El listado de actividades (RF-01, RF-05) y las búsquedas (RF-07, RF-08) deben ser rápidos.                                          |
| 4 | Uso Intensivo del Entorno Operativo | 2                | Usa herramientas estándar, pero el chatbot podría requerir APIs externas.                                                           |
| 5 | Tasa de Transacción                 | 3                | Varias transacciones por hora (Inscripción, login, edición) son importantes.                                                        |
| 6 | Entrada de Datos Online             | 5                | Todas las entradas (Registro, Login, Edición) son en línea y esenciales.                                                            |
| 7 | Eficiencia del Usuario Final        | 4                | El sistema debe ser altamente eficiente (buena UX) para Socio e Instructor.                                                         |
| 8 | Actualización Online                | 5                | El registro, inscripción y edición de perfil (RF-03, RF-10, RF-13) son vitales y se actualizan de inmediato.                        |



|    |                                       |   |                                                                                                              |
|----|---------------------------------------|---|--------------------------------------------------------------------------------------------------------------|
| 9  | Complejidad del Procesamiento Interno | 4 | La lógica de validación (cupos, permisos, estado ACTIVE/INACTIVE) y los gráficos (RF-33) añaden complejidad. |
| 10 | Reusabilidad                          | 2 | Se reutiliza lógica entre RF-03 y RF-15, lo que le da una influencia media-baja.                             |
| 11 | Facilidad de Instalación              | 2 | Instalación sencilla si se usa Docker/contenedores.                                                          |
| 12 | Facilidad de Operación                | 3 | Requiere monitoreo de administradores.                                                                       |
| 13 | Múltiples Sitios                      | 0 | Asumimos que opera en un único centro.                                                                       |
| 14 | Facilidad de Cambio                   | 3 | Se espera que el sistema permita cambios en tarifas o actividades con facilidad.                             |

$$\text{SGI} = \sum \text{GI} = 41$$

$$\text{FA} = 0.65 + (0.01 \times \text{SGI})$$

$$\text{FA} = 0.65 + (0.01 \times 41)$$

$$\text{FA} = 0.65 + 0.41$$

$$\text{FA} = 1.06$$

$$\text{PFA} = \text{UFP} \times \text{FA} = 137 \times 1.06 = 145.22$$

## ESTIMACIÓN DE ESFUERZO, TIEMPO Y COSTO

### Técnica Obligatoria 1: **COCOMO**

$$\text{LOC} = \text{PFA} \times \text{Factor de Conversión}$$

$$\text{LOC} = 145.22 \times 55$$

$$\text{LOC} = 7,987 \text{ LOC}$$

$$\text{KLOC} \approx 7.99 \text{ KLOC}$$

Tipo de proyecto: **Orgánico**

## ESFUERZO

$$PM = A * (KSLOC) ^ B$$

$$PM = 2.4 * (7.99) ^ 1.05$$

$$PM = 21.29 \text{ personas-mes}$$

## TIEMPO

$$TDEV = C * (PM) ^ D$$

$$TDEV = 2.5 * (21.29) ^ 0.38$$

$$TDEV = 8.3 \text{ meses}$$

## PERSONAS PROMEDIO

$$N = 21.29 / 8.3$$

$$N = 2.56 \text{ personas}$$

**Observación.** Para el Costo Promedio Mensual por Persona utilizamos un valor estimado de 1000 USD/Mes.

## COSTO TOTAL

$$C = (2.56 * 1000 * 8.3)$$

$$C = 21.248 \text{ USD}$$

## Explicación de la Elección del Tipo de Proyecto

Se eligió el modo Orgánico para la estimación del proyecto por las siguientes características que se ajustan al contexto:

1. **Tamaño del Proyecto (KLOC):** El proyecto es de tamaño pequeño a mediano, con una estimación de 7.99 KLOC. Los proyectos orgánicos típicamente se encuentran por debajo de 50 KLOC.
2. **Entorno y Experiencia del Equipo:** Nuestro equipo de desarrollo es reducido (dos personas) y contamos con una experiencia moderada en la tecnología elegida (Java/Spring Boot). Trabajamos en un entorno de desarrollo académico, con presión y plazos de entrega que no son tan rígidos como en un proyecto comercial a gran escala.
3. **Complejidad del Problema:** El sistema es una aplicación de gestión de un centro deportivo (CRUDs, lógica de inscripción, manejo de roles) bien definida, sin requisitos de *hardware* complejo, ni algoritmos altamente novedosos.



## Supuestos utilizados

| Categoría                | Supuesto                                                                                                                                                                   | Valor Utilizado              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Punto de Partida         | Se utilizó la medición funcional ajustada de Puntos de Función Ajustados (PFA)                                                                                             | 145.22 PFA                   |
| Factor de Conversión     | El factor de conversión de PF a Líneas de Código (LOC) para el lenguaje de <i>backend Java</i> es de <b>55 LOC/PF</b> .                                                    | 55 LOC/PFA                   |
| Alcance de Estimación    | La estimación de KLOC se aplica únicamente al código del <b>Backend</b> (Java/Spring Boot), ya que contiene la lógica de negocio central medida por los Puntos de Función. | 7.99 KLOC                    |
| Coeficientes COCOMO      | Se utilizan los coeficientes del modo <b>Orgánico</b>                                                                                                                      | A=2.4, B=1.05, C=2.5, D=0.38 |
| Costo Laboral            | El costo total se calcula utilizando un costo promedio mensual por recurso humano.                                                                                         | 1000 USD/persona-mes         |
| Disponibilidad de Equipo | Para el cálculo teórico del modelo, se utiliza el supuesto estándar de <b>dedicación al 100% (tiempo completo)</b> .                                                       | 2,5 personas                 |

## Técnica Obligatoria 2: **PLANNING POKER**

### Contexto:

Equipo: Matías Benedetti (Scrum Master y desarrollador), Tomás Bassi (Product Owner y desarrollador).

Fecha: 28/05/2025

Duración: 2 horas

RF-01: El usuario anónimo puede consultar el listado de todas las actividades disponibles.

RF-05: El socio puede consultar todas las actividades disponibles.

RF-18: El instructor puede ver el listado de todas las actividades del sistema.

Ronda 1:

- Matías: 3 -> "No es muy complejo"
- Tomás: 3 -> "Estoy de acuerdo, ya hemos hecho varios"

Consenso: 3 Story Points

RF-02: El usuario anónimo puede visualizar el detalle de una actividad seleccionada, incluyendo horario, instructor y precio.

RF-06. El socio puede ver el detalle de una actividad.

RF-20. El instructor puede ver el detalle de una actividad.

Ronda 1:

- Tomás: 1 -> "Es una consulta simple. Solo exponer un DTO con la info por ID"
- Matías: 1 -> "Coincido. Un solo *endpoint* GET."

Consenso: 1 Story Point

RF-03: El usuario anónimo puede registrarse como socio.

Ronda 1:

- Matías: 5 -> "Requiere validaciones de formato para todos los campos, y determinación automática de Status: INACTIVE para el usuario."
- Tomás: 5 -> "Si, puede llevar un rato, porque también hay que verificar la unicidad de los datos."

Consenso: 5 Story Points

RF-04: El usuario puede iniciar sesión.

Ronda 1:

- Matías: 5 -> "Es medio complejo, requiere validaciones y JWT"
- Tomás: 8 -> "Nunca implemente JWT"

Discusión

- Matías se ofrece para explicarle a Tomás.

Consenso: 5 Story Points.

RF-07. El socio puede buscar actividades por nombre.

RF-08. El socio puede buscar actividades por rango horario.

Ronda 1:

- Matías: 5 -> "Son dos filtros, uno por nombre y otro por rango horario."
- Tomás: 8 -> "Si, quizás la búsqueda por rango horario puede costar un poco más."

Consenso: 5 Story Points.

RF-09. El socio puede ver el detalle del instructor que dicta una actividad.

Consenso: 3 Story Points.

RF-10. El socio puede inscribirse a una actividad.

RF-21. El instructor puede inscribir un socio en una actividad que dicta.

RF-27. El administrador puede inscribir a un socio en cualquier actividad.

Ronda 1:

- Tomás: 3 -> "Requiere tres validaciones críticas en el backend: Cupo Máximo, que

el socio no esté ya inscrito, y actualizar el Status del socio a ACTIVE si estaba INACTIVE”

- Matías: 3 -> “Si, y manejar todas las excepciones.”

Consenso: 3 Story Points.

RF-11. El socio puede ver el listado de actividades en las que está inscripto.

Consenso: 3 Story Points.

RF-12. El socio puede darse de baja de una actividad.

RF-22. El instructor puede dar de baja a un socio de una actividad que dicta.

RF-28. El administrador puede dar de baja a un socio de cualquier actividad.

Ronda 1:

- Tomás: 3 -> “Es una actualización de estado del registro de Enrollment (dar de baja) y no de eliminación total. Es un endpoint sencillo.”
- Matías: 3 -> “Si, la lógica es mínima.”

Consenso: 2 Story Points.

RF-13. El socio puede editar su información personal.

Ronda 1:

- Matías: 5 -> “Es un endpoint PUT con validación de unicidad de Email (si se modifica) y manejo de actualización parcial de datos.”
- Tomás: 8 -> “Si, y también con autocompletado de formulario.”

Consenso: 3 Story Points.

RF-14. El socio puede eliminar su cuenta.

Ronda 1:

- Matías: 3 -> “Hay que borrar *Enrollments* asociados.”

Consenso: 2 Story Points.

RF-15. El instructor puede registrar nuevos socios.

Consenso: 2 Story Points.

RF-16. El instructor puede listar a todos los socios.

Ronda 1:

- Tomás: 3 -> “Es un listado paginado simple de usuarios con rol MEMBER. No hay lógica de filtros complejos.”

Consenso: 2 Story Points.

RF-17. El instructor puede ver el detalle de un socio.

Consenso: 3 Story Points

RF-25. El administrador puede ver el detalle de cualquier usuario y actividad.

Consenso: 2 Story Points.

RF-19. El instructor puede ver el listado de actividades que dicta.

Consenso: 3 Story Points.

RF-23. El administrador puede crear socios, instructores y otros administradores.

Ronda 1:

- Matías: 5 -> "Son tres tipos de usuarios diferentes. Cada tipo tiene validaciones y campos específicos"
- Tomás: 5 -> "El formulario cambia dinámicamente según el tipo. Aunque la base es la misma."

Consenso: 5 Story Points.

RF-24. El administrador puede listar todos los usuarios (socios, instructores, administradores).

Ronda 1:

- Matías: 5 -> "Es un listado con paginación, filtros por rol y búsqueda".

Consenso: 3 Story Points.

RF-26. El administrador puede eliminar socios e instructores.

Ronda 1:

- Tomás: 3 -> "Parecido al de eliminación de cuenta pero con mas validaciones."

Consenso: 3 Story Points.

RF-29. Todos los usuarios pueden crear rutinas.

Ronda 1:

- Matías: 5 -> "Es un formulario complejo con días y ejercicios anidados."
- Tomás: 5 -> "Hay que manejar una segunda base de datos."

Consenso: 5 Story Points.

RF-30. El instructor puede asignar rutinas.

Ronda 1:

- Matías: 5 -> "Solo es un endpoint POST que asocia un ID de Member con un ID de Routine."
- Tomás: 5 -> "Mínima lógica de validación, solo existencia de IDs."

Consenso: 2 Story Points.

RF-31. El socio puede ver su rutina.

Consenso: 2 Story Points.

RF-32. El socio puede cargar su progreso.

Ronda 1:

- Matías: 3 -> "Tiene que guardar peso, repeticiones y fecha por cada ejercicio."
- Tomás: 3 -> "El formulario es muy dinámico y hay que validar el formato de entrada."

Consenso: 3 Story Points.

RF-33. El socio puede ver gráficos con su progreso por ejercicio por rutina.

Ronda 1:

- Matías: 5 -> "La consulta a la base de datos es simple, pero requiere integración con una librería de gráficos (Chart.js) en Angular. La visualización de datos añade complejidad de frontend."

Consenso: 3 Story Points.

RF-34. El socio puede eliminar su rutina.

Consenso: 3 Story Points.

RF-35. El instructor puede editar sus rutinas.

Consenso: 5 Story Points.

RF-36. El usuario anónimo puede interactuar con el chatbot para consultar información general sobre actividades.

RF-37. El socio puede usar el chatbot para consultar información personalizada sobre sus inscripciones.

RF-38. El chatbot puede proporcionar información sobre horarios, instructores y disponibilidad de cupos de las actividades.

RF-39. El chatbot puede responder preguntas en lenguaje natural sobre las actividades del centro deportivo.

Ronda 1:

- Tomás: 8 -> "Nunca trabajamos con APIs de IA"
- Matías: 5 -> "Hay que enviar contexto y procesar respuestas."

Discusión:

Tomas se encuentra preocupado sobre la curva de aprendizaje y tiempos. Matias investiga y encuentra que Gemini tiene SDK para Java.

Consenso: 5 Story Points.

| Métrica                 | Valor Confirmado | Justificación                                                                              |
|-------------------------|------------------|--------------------------------------------------------------------------------------------|
| Total Story Points      | 125 SP           | Suma del esfuerzo de los 39 RFs.                                                           |
| Factor de Conversión    | 1 SP = 3 Horas   | Factor de alta productividad asumido por el equipo para el esfuerzo de desarrollo.         |
| Jornada Efectiva Diaria | 3 horas / día    | Dedicación promedio realista del equipo (2 estudiantes) considerando compromisos externos. |
| Días Laborables/Mes     | 22 días/mes      | Estándar de días laborables por mes                                                        |

#### Cálculos:

- Esfuerzo Total (Horas)**

Esfuerzo Total (h) = Total SP x Factor de Conversión

Esfuerzo Total (h) = 125 SP x 3 h/SP = 375 horas

- Tiempo en Días Laborables**

Días Laborables Totales = Esfuerzo Total (h) / Jornada Efectiva (h/día)

Días Laborables Totales = 375 h / 3 h/día = 125 días

- Resultado Final de la estimación**

Tiempo en Meses = Días Laborables Totales / Días Laborables/mes

Tiempo en Meses = 125 días / 22 días/mes ≈ **5.68 meses**

### Comparación

#### Resultados Clave

| Métrica                     | COCOMO Básico (Formal) | Planning Poker (Ágil) |
|-----------------------------|------------------------|-----------------------|
| Punto de Partida            | KLOC (7.99 KLOC)       | Story Points (125 SP) |
| Esfuerzo Total              | 21.29 personas-mes     | 375 horas             |
| Tiempo de Desarrollo (TDEV) | 8.3 meses              | 5.68 meses            |

### Similitudes entre COCOMO y Planning Poker

Ambas técnicas cumplen el objetivo fundamental de la estimación de proyectos, aunque desde enfoques diferentes:

- **Fundamento en la Complejidad:** Ambas estimaciones parten de la complejidad funcional del sistema (medida por PFA en COCOMO y SP en Planning Poker). No son estimaciones basadas en el tiempo realmente transcurrido, sino en el esfuerzo necesario.
- **Escalabilidad:** Ambas utilizan escalas no lineales para reflejar la complejidad. Planning Poker usa la serie de Fibonacci y COCOMO utiliza una curva exponencial ( $KLOC^B$ ) que refleja que proyectos más grandes requieren un esfuerzo mayor.
- **Consistencia de Unidad:** Ambas convierten su unidad de complejidad (KLOC o SP) a una unidad de esfuerzo utilizable (personas-mes o horas), permitiendo la planificación de recursos.

## Diferencias Clave

Las diferencias radican en la forma de estimación, la inclusión de actividades de ciclo de vida, y el enfoque del equipo:

| Característica   | COCOMO Básico                                                                                                | Planning Poker                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Enfoque          | Técnico-Matemático. Basado en el tamaño del producto (KLOC).                                                 | Basado en la experiencia y la opinión colectiva del equipo.                                                   |
| Alcance          | Ciclo de Vida Completo. Incluye actividades de gestión, integración, testing, y documentación formal.        | Desarrollo Puro. Se enfoca en el esfuerzo de programación, diseño y pruebas unitarias de una <i>feature</i> . |
| Ajuste           | Externo y Formal. Se ajusta mediante el Factor de Ajuste (FA) y la elección del Modo de Proyecto (Orgánico). | Interno y Discutido. El consenso y la discusión entre el PO y el SM/Developer son el mecanismo de ajuste.     |
| Unidad de Medida | PFA -> KLOC -> Personas-Mes (PM)                                                                             | Requisito -> Story Point (SP) -> Horas/Días                                                                   |

## Análisis de la Diferencia en el Tiempo de Desarrollo

La diferencia entre el tiempo estimado por COCOMO (8.3 meses) y Planning Poker (5.68 meses) se debe principalmente al alcance del esfuerzo que mide cada técnica:

### COCOMO (8.3 meses): Esfuerzo de Proceso Total

Como modelo paramétrico, estima el esfuerzo necesario para el ciclo de vida completo del proyecto (incluyendo actividades funcionales y no funcionales):

- Gestión y Planificación: El tiempo que el Scrum Master dedica a la facilitación.
- Documentación Formal: Generación de manuales y documentos de arquitectura.
- Integración y Pruebas de Sistema: El tiempo dedicado a la prueba de extremo a extremo del sistema.

### Planning Poker (5.68 meses): Esfuerzo de Desarrollo Puro



Solo mide el esfuerzo de implementación y codificación de un requisito.

- No incluye las horas dedicadas a tareas de gestión, reuniones, revisiones de código que no se mapean a una historia específica, o el esfuerzo administrativo.
- El uso del factor de 1 SP = 3 horas es quizás un poco optimista, puede variar, pero usamos un promedio aproximado.

## Conclusión

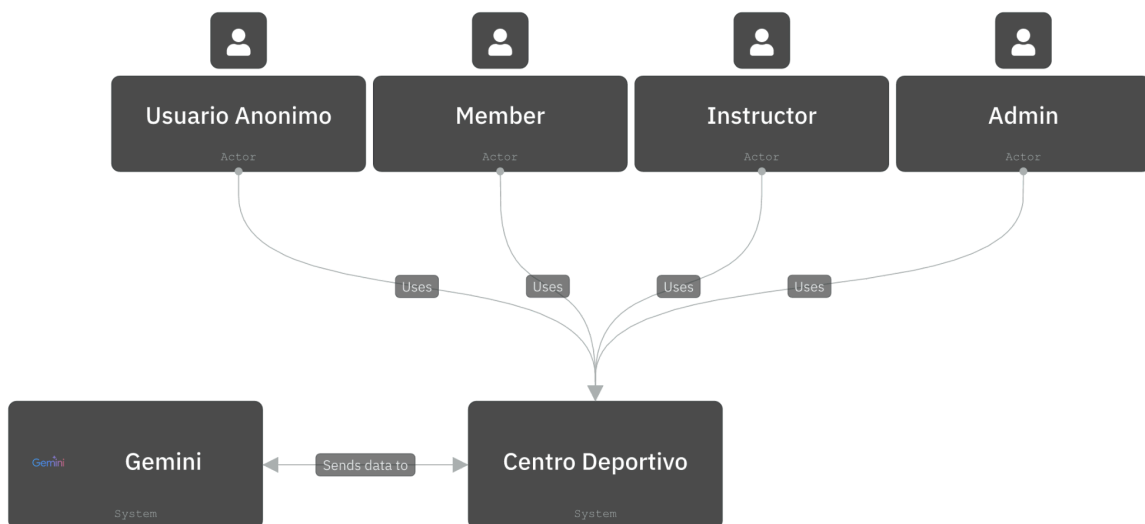
La estimación de COCOMO Básico es la cifra más adecuada para el tiempo total del proyecto desde el inicio hasta la entrega, ya que incluye todas las actividades necesarias.

La estimación de Planning Poker es la cifra más útil para la planificación ágil, ya que representa el tiempo que el equipo necesita para construir el código funcional.

## Arquitectura del sistema - Modelo C4

[Diagramas en IcePanel](#)

### Nivel 1: Diagrama de contexto



### Descripción

El sistema es una aplicación web que permite gestionar actividades deportivas, usuarios e inscripciones en un centro deportivo. Conecta a usuarios anónimos, socios, instructores y administradores para consultar, gestionar e inscribirse en actividades. Integra un chatbot con IA para asistencia informativa.

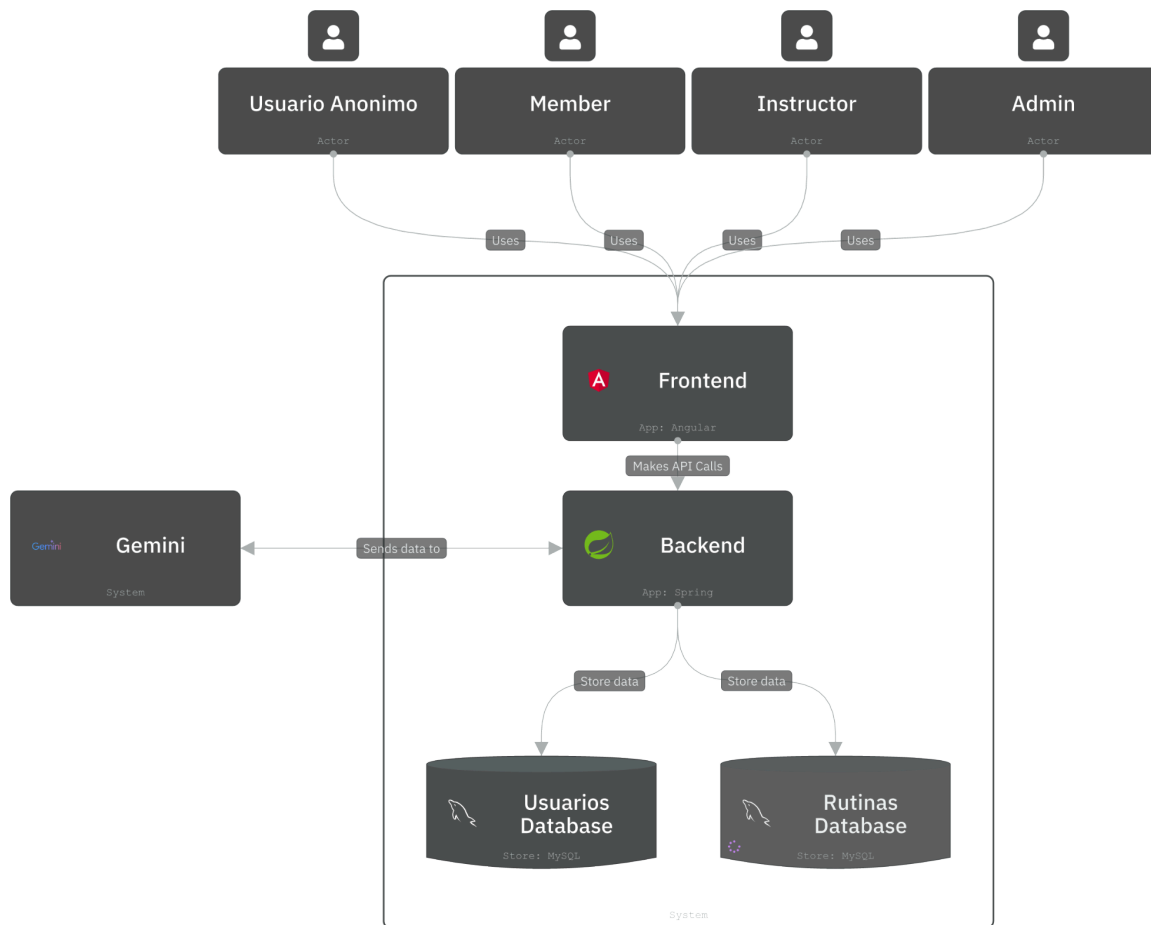
## Tecnologías

- FrontEnd: Angular 20 (TypeScript, RxJS, Chart.js)
- BackEnd: Spring Boot 3.4.5, Spring Security, Spring Data JPA
- Base de Datos: MySQL
- Autenticación: JWT
- Documentación API: OpenAPI/Swagger

## Responsabilidades

- Usuario Anónimo (Actor): Consultar actividades disponibles, ver detalles de actividades e instructores, registrarse como socio, iniciar sesión.
- Member/Socio (Actor): Gestionar su perfil, buscar actividades por nombre o horario, inscribirse y darse de baja de actividades, consultar sus inscripciones.
- Instructor (Actor): Registrar nuevos socios, listar y consultar socios, ver todas las actividades, gestionar inscripciones en sus actividades.
- Administrador (Actor): Crear usuarios (socios, instructores, administradores), gestión total de usuarios, inscribir o dar de baja socios en cualquier actividad, control granular mediante niveles de permiso.
- Centro Deportivo: Proporciona UI web, autentica usuarios con JWT, gestiona usuarios y actividades, controla inscripciones y cupos, almacena datos en MySQL, integra chatbot con IA.
- Gemini API: Provee servicio de chatbot con inteligencia artificial para responder consultas sobre actividades, horarios e inscripciones.

## Nivel 2: Diagrama de contenedores



## Descripción

El sistema se divide en un frontend SPA que interactúa con una API REST backend. El backend gestiona la lógica de negocio, autenticación JWT, integración con chatbot IA y persiste datos en MySQL. Existe una segunda base de datos planificada para rutinas.

## Responsabilidades

**Frontend Web (Angular SPA):** Proporciona UI responsive, gestiona navegación y rutas, autentica usuarios con JWT, consume endpoints REST del backend, renderiza visualizaciones, implementa guards de autorización por roles.

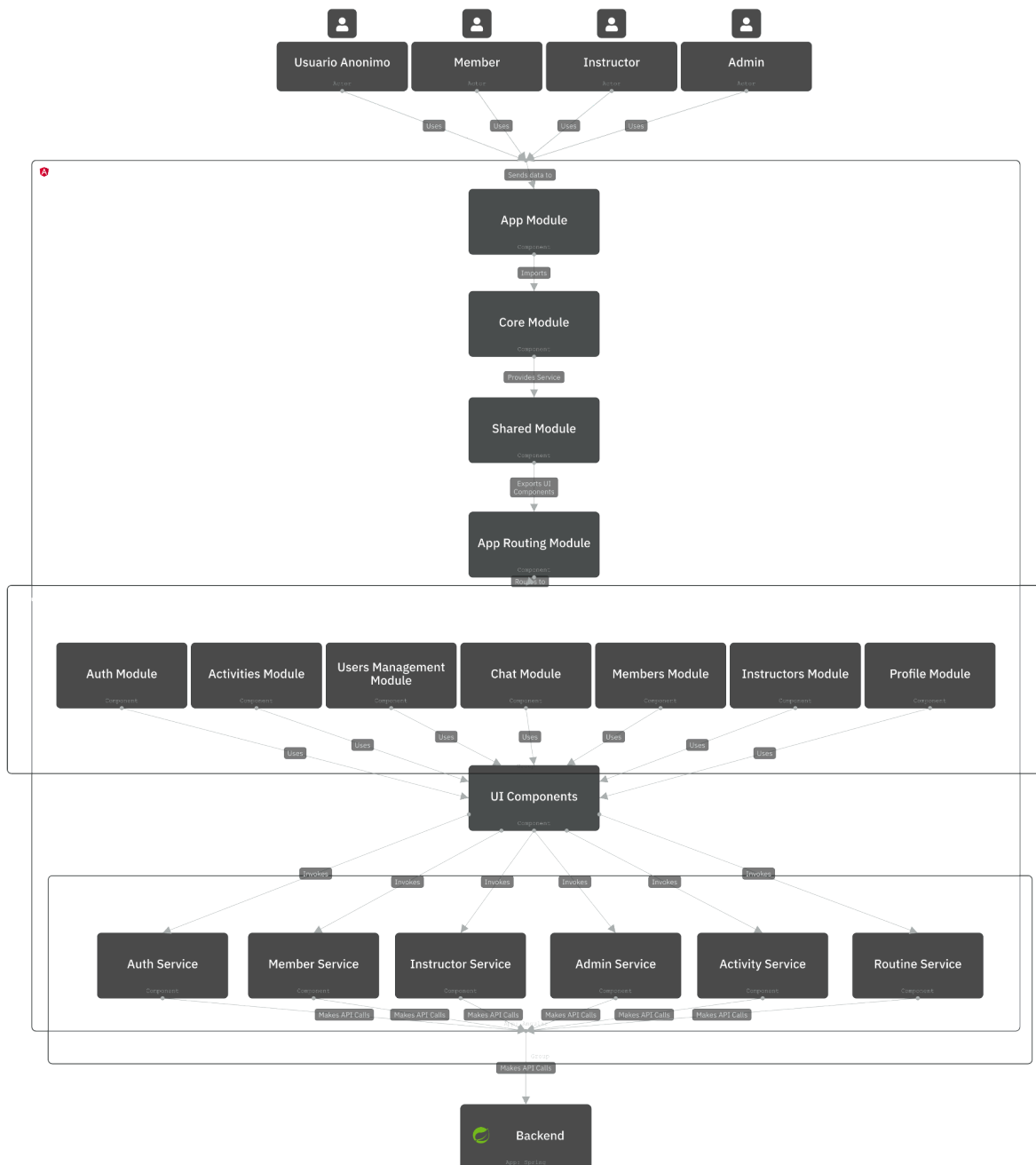
**Backend API REST (Spring Boot):** Expone endpoints REST, autentica y autoriza con Spring Security y JWT, valida permisos por roles, implementa lógica de negocio, gestiona inscripciones y actividades, procesa chatbot con GeminiService, se comunica con Gemini API, persiste datos mediante JPA/Hibernate.

**Base de Datos Principal (MySQL):** Almacena entidades User (Member, Instructor, Admin), SportActivity, Enrollment, Credential, gestiona relaciones y autenticación, esquema actualizado por Hibernate.

Base de Datos Rutinas (MySQL - Futura): Almacenará rutinas de entrenamiento personalizadas (integración planificada).

## Nivel 3: Diagrama de componentes

### Frontend

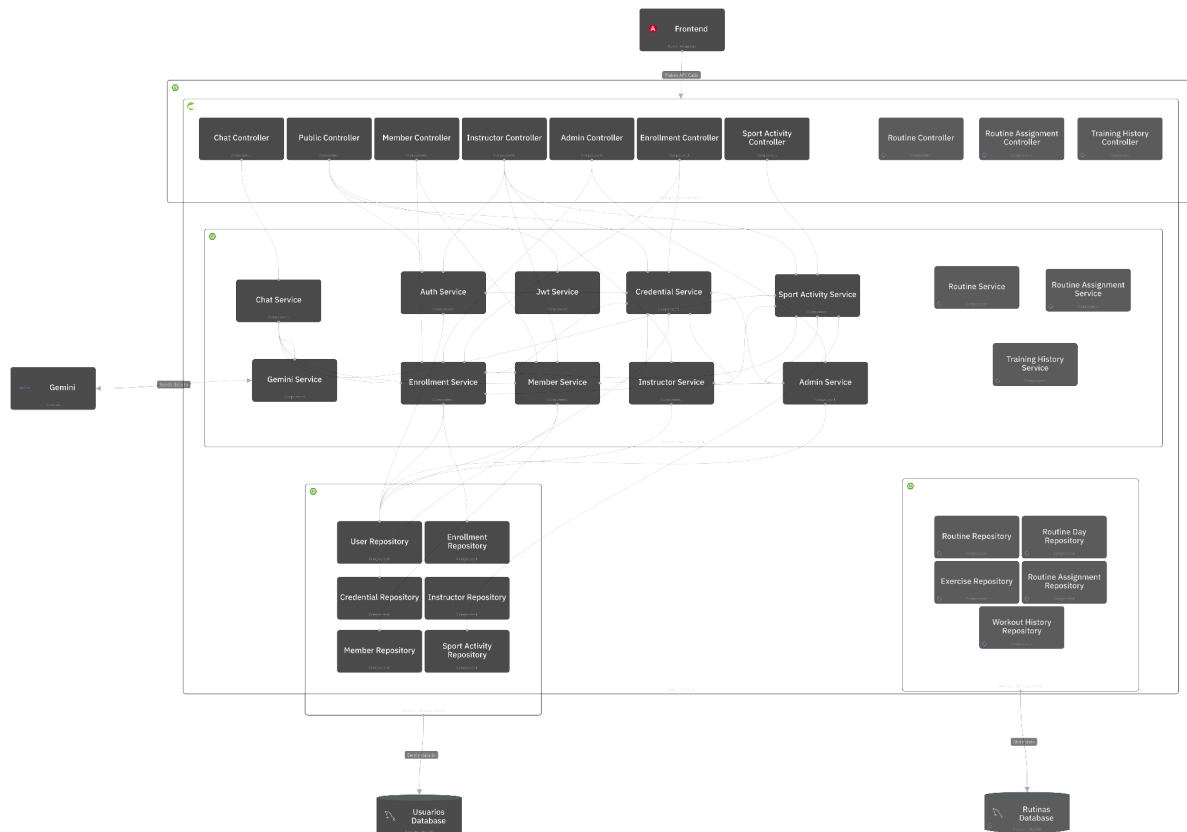


### Responsabilidades

- Features Module: Funcionalidades de cada dominio, sus componentes y rutas
- Core Module: Contiene interceptor jwt y guards.

- Shared Module: Contiene componentes, modelos, servicios, store y validadores globales.
- App Routing Module: Define y gestiona navegación entre vistas y módulos.
- UI Module: Renderiza las interfaces visuales.
- Services: Manejan la lógica de comunicación con el back y aplican las reglas de negocio definidas en el back.
- App Module: Coordina interacciones del usuario con los servicios y las vistas.

## Backend



## Responsabilidades

- Controller: Recibe solicitudes HTTP, expone endpoints que consume el Front-End, valida los datos de entrada y devuelve respuestas en formato de JSON.
- Service: Se encarga de las reglas de negocio que debe cumplir la información.
- Repository: Se comunica con la base de datos, provee acceso a la misma, maneja consultas y mapea datos a las entidades.
- Spring Security: Autenticar usuarios, crear tokens y validarlos, autorizar según rol.

## Gestión de riesgos

| ID   | Riesgo                                                                                            | Categoría    | Probabilidad | Impacto | Exposición | Estrategia                                                                                                                                | Responsable   |
|------|---------------------------------------------------------------------------------------------------|--------------|--------------|---------|------------|-------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| R-01 | Fallas en la integración entre Spring Boot y Angular debido a contratos de API inconsistentes     | Tecnico      | Alta         | Alto    | 25         | Mitigar: Definir DTOs y contratos de API, documentar endpoints con Swagger                                                                | Tomas         |
| R-02 | Problemas de rendimiento en búsquedas complejas                                                   | Tecnico      | Media        | Medio   | 9          | Mitigar: Optimizar queries con JPA, implementar paginación en todos los listados                                                          | Matias        |
| R-03 | Vulnerabilidades de seguridad en implementación de JWT o manejo de credenciales                   | Tecnico      | Baja         | Alto    | 10         | Mitigar: Usar librería Spring Security, revisar configuración de CORS                                                                     | Matias        |
| R-04 | Cambios en requisitos funcionales durante el desarrollo por feedback del cliente                  | Alcance      | Alta         | Medio   | 20         | Evitar/Mitigar: Documentar RF aprobados, registrar cambios en backlog priorizado                                                          | Tomas, Matias |
| R-05 | Baja disponibilidad del equipo por parciales, trabajos de otras materias y compromisos personales | Equipo       | Alta         | Medio   | 20         | Mitigar: Planificar sprints considerando calendario académico, mejor redistribución de tareas, mantener comunicación sobre disponibilidad | Tomas, Matias |
| R-06 | Falta de conocimiento técnico en tecnologías específicas                                          | Equipo       | Media        | Medio   | 9          | Mitigar: Compartir documentación técnica, reuniones para analizar y desbloquear impedimentos                                              | Tomas         |
| R-07 | Dependencia de servicios externos (Google Gemini)                                                 | Programático | Baja         | Media   | 6          | Aceptar: Tener respuestas para posibles fallas o                                                                                          | Matias        |

|  |     |  |  |  |  |                                         |  |
|--|-----|--|--|--|--|-----------------------------------------|--|
|  | AI) |  |  |  |  | latencia en el uso del servicio externo |  |
|--|-----|--|--|--|--|-----------------------------------------|--|

## Decisiones claves

### ADR 001: Integración de Chatbot conversacional con IA mediante Google Gemini

#### Estado

- Estado: Aceptado
- Fecha: 2025-11-08
- Decisor/es: Equipo de desarrollo
- Sustituye a (si aplica): N/A

#### Contexto

El proyecto del Centro Deportivo requiere proporcionar asistencia inmediata a los usuarios para consultar información sobre actividades disponibles, horarios, instructores, inscripciones personales y disponibilidad de cupos. Los usuarios (públicos, socios e instructores) necesitan una forma rápida de obtener información sin necesidad de navegar por múltiples páginas o contactar al personal del centro.

Se identificó la necesidad de implementar un asistente virtual que:

- Responda preguntas en lenguaje natural sobre las actividades del centro deportivo.
- Proporcione información personalizada según el usuario autenticado.
- Esté disponible 24/7 sin intervención humana.
- Acceda a datos en tiempo real del sistema (actividades, inscripciones, disponibilidad).
- Mantenga un contexto conversacional apropiado y evite dar consejos médicos.

#### Alternativas consideradas

| Alternativa                      | Pros                                                                                                                                              | Contras                                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Google Gemini API (Seleccionado) | Modelo de lenguaje potente (Gemini 2.5 Flash). API REST sencilla de integrar. Capa gratuita generosa. Baja latencia. Soporte nativo para español. | Requiere conexión a internet. Dependencia de servicio externo de Google. Costos potenciales al escalar. |
| OpenAI GPT-4/GPT-3.5             | Muy potente y ampliamente                                                                                                                         | Costos más altos que                                                                                    |

|                                   |                                                                                                        |                                                                                                                                                               |
|-----------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                   | utilizado. Excelente comprensión del lenguaje natural. APIs bien documentadas.                         | Gemini. Sin capa gratuita significativa. Mayor latencia en algunas regiones.                                                                                  |
| Chatbot basado en reglas (sin IA) | Sin dependencias externas. Costos nulos. Totalmente predecible. Control completo sobre las respuestas. | Inflexible y limitado. Requiere definir cada posible interacción. Experiencia de usuario pobre ante variaciones del lenguaje. No escala con nuevas preguntas. |

## Decisión

Se elige Google Gemini API (modelo Gemini 2.5 Flash) como proveedor de inteligencia artificial generativa para el chatbot del Centro Deportivo, llamado ChatBum.

La integración se realizará mediante:

- Un servicio backend (ChatService y GeminiService) que construye contexto personalizado con datos en tiempo real del sistema y realiza llamadas HTTP a la API de Gemini.
- Un widget flotante (ChatWidget) en el frontend Angular que permite la interacción conversacional.
- Persistencia del historial de conversación en localStorage del navegador para mantener el contexto entre sesiones.
- Sistema de prompts estructurados que incluyen: fecha/hora actual, listado de actividades, información del usuario autenticado, inscripciones activas y reglas de comportamiento (como no dar consejos médicos).

## Consecuencias

### Impactos positivos

- Experiencia de usuario mejorada: Los usuarios obtienen respuestas instantáneas en lenguaje natural sin necesidad de buscar en la interfaz o contactar al personal.
- Personalización: El chatbot accede a las inscripciones y perfil del usuario autenticado, brindando respuestas contextualizadas.
- Disponibilidad 24/7: El asistente está disponible en todo momento sin costos de personal.
- Integración completa: El chatbot consume datos en tiempo real del backend (actividades, instructores, cupos, inscripciones), garantizando información actualizada.
- Facilidad de implementación: La API REST de Gemini es sencilla de integrar con el stack tecnológico existente (Spring Boot + Angular).
- Capa gratuita: Google ofrece una cuota gratuita generosa que cubre las necesidades del proyecto en etapa inicial.



## Impactos negativos

- Dependencia externa: El sistema depende de la disponibilidad y estabilidad del servicio de Google Gemini. Si la API está caída o con problemas, el chatbot no funcionará.
- Costos potenciales: El uso intensivo puede generar costos mensuales.
- Latencia de Red: Las respuestas dependen de llamadas HTTP externas, lo que puede introducir latencia.
- Seguridad de API Key: Se requiere gestionar de forma segura la GEMINI\_API\_KEY mediante variables de entorno para evitar exposición en el código.
- Respuestas impredecibles: Aunque se usan prompts estructurados, la IA puede generar respuestas inesperadas o incorrectas.
- Complejidad en prompts: Mantener y actualizar los prompts con contexto detallado requiere esfuerzo continuo a medida que el sistema evoluciona.
- Filtros de seguridad: Gemini puede bloquear respuestas consideradas sensibles (ej: temas de salud), requiriendo manejo especial de estos casos.

## ADR 002: Selección de arquitectura de persistencia para el módulo de rutinas de entrenamiento

### Estado

- Estado: Aceptado
- Fecha: 2025-11-13
- Decisor/es: Equipo de desarrollo
- Sustituye a: N/A

### Contexto

El proyecto Centro Deportivo requiere implementar un módulo de rutinas de entrenamiento donde instructores puedan crear rutinas completas (con días, ejercicios, series y repeticiones) y asignarlas a socios, mientras que los socios puedan crear sus propias rutinas personales y registrar su progreso histórico.

Necesitábamos decidir entre dos arquitecturas principales: mantener toda la lógica en el backend actual de Spring Boot con una base de datos relacional separada, o utilizar Supabase (PostgreSQL + API REST autogenerada) como backend alternativo para el módulo de rutinas. La decisión afecta directamente la complejidad del desarrollo, el tiempo de implementación, la consistencia arquitectónica del proyecto y la curva de aprendizaje del equipo.

El sistema actual ya cuenta con Spring Boot como backend consolidado, MySQL como motor de base de datos, y autenticación JWT implementada. El equipo tiene experiencia en este stack tecnológico pero no tiene conocimientos previos de Supabase.

### Alternativas consideradas

| Alternativa                                 | Pros                                                                                                                                                                                                                                        | Contras                                                                                                                                                                                              |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Spring Boot + MySQL separado (seleccionado) | Mantiene consistencia tecnológica con el proyecto actual. Equipo ya familiarizado con Spring Boot y MySQL. Control total sobre la lógica de negocio y queries. Fácil evolución futura a microservicios. No introduce dependencias externas. | Mayor cantidad de código a desarrollar. Tiempo de desarrollo más largo. Requiere implementar toda la capa de servicios manualmente.                                                                  |
| Supabase (PostgreSQL + API REST)            | Desarrollo más rápido (API autogenerada). Menor cantidad de código a escribir.                                                                                                                                                              | Introduce nueva tecnología al stack. Arquitectura híbrida con dos backends diferentes. Dependencia de servicio externo. Equipo sin experiencia previa. Menor control sobre queries y optimizaciones. |
| Spring Boot + Base de Datos Única           | Menor complejidad de configuración. Queries optimizadas con joins nativos.                                                                                                                                                                  | Alto acoplamiento entre dominios. Dificulta la escalabilidad futura. Cambios en rutinas pueden afectar al sistema principal. No permite evolución a microservicios.                                  |

## Decisión

Se elige Spring Boot con una base de datos MySQL separada para el desarrollo del módulo de rutinas.

Esta decisión se fundamenta en mantener la consistencia tecnológica del proyecto, aprovechar el conocimiento existente del equipo en Spring Boot y evitar la introducción de nuevas tecnologías que requerirían capacitación adicional. Aunque el tiempo de desarrollo es mayor, priorizamos la cohesión arquitectónica, el control total sobre la implementación y la preparación del sistema para una futura evolución hacia microservicios. La separación en bases de datos distintas nos permite bajo acoplamiento entre dominios sin sacrificar la uniformidad del stack tecnológico.

## Consecuencias

### Impactos positivos

- Consistencia arquitectónica: Todo el sistema utiliza el mismo stack tecnológico (Spring Boot + MySQL), facilitando el mantenimiento y la incorporación de nuevos desarrolladores.

- Conocimiento del equipo: Aprovechamos la experiencia existente en Spring Boot, JPA y MySQL, evitando la curva de aprendizaje de Supabase y sus particularidades (Row Level Security, API autogenerada, PostgreSQL-specific features).
- Control total: Tenemos control completo sobre la lógica de negocio, queries, optimizaciones y estructura de datos.
- Bajo acoplamiento con separación de BDs: La arquitectura de bases de datos separadas reduce el acoplamiento entre el módulo de rutinas y el sistema principal, facilitando una futura extracción como microservicio si el proyecto escala.

### Impactos negativos

- Mayor tiempo de desarrollo: Estimamos varias semanas más en lugar de las pocas que tomaría con Supabase, debido a la necesidad de implementar manualmente toda la capa de servicios, controllers, validaciones, DTOs y mappers.
- Validaciones manuales de integridad: Al no tener FKs reales entre bases de datos (solo username como FK lógica), debemos implementar todas las validaciones de integridad referencial en código, con el riesgo de datos huérfanos si no se implementa correctamente.

## SCRUM

Durante la gestión del proyecto se implementa la metodología ágil SCRUM. Las ceremonias principales, sprint planning, sprint review, sprint retrospective, se realizan durante las horas de la materia.

### Definición de roles

**Product Owner [Bassi Servant, Tomas]:** Se encargará de la gestión del Product Backlog a través de la herramienta JIRA, lo que implica la definición y priorización de las historias de usuario (requisitos). Además, será el enlace principal con los docentes, quienes asumen el rol de cliente para la validación.

**Scrum Master [Benedetti, Matias]:** Guiará al equipo en las prácticas de SCRUM, moderará las reuniones de planificación, revisión y retrospectiva en clase, y gestionará la resolución de cualquier bloqueo.

### Product Backlog

Se compone de la lista priorizada de Épicas (EPIC) e Historias de Usuario (US), las cuales se gestionan y refinan de manera continua. Para la estimación de cada ítem del backlog, el equipo ha adoptado la siguiente metodología:

- **Técnica de Estimación:** Se utiliza la técnica de Puntos de Historia (Story Points) para valorar de forma relativa el esfuerzo, la complejidad y la incertidumbre de cada Historia de Usuario.
- **Secuencia de Estimación:** La asignación de estos puntos se realiza utilizando la secuencia de Fibonacci (ej: 1, 2, 3, 5, 8, 13...). Esta escala no lineal ayuda al equipo a reflejar mejor la incertidumbre inherente a las tareas más grandes y complejas.

- Herramienta de Gestión: Toda la gestión del Product Backlog, la asignación de Story Points a cada historia y el seguimiento de los Sprints se centraliza en la herramienta Jira.

## EPIC/Gestión de acceso

- US/Registro de socios
- US/Inicio de sesión

## EPIC/Catálogo de actividades

- US/Listado de actividades
- US/Búsqueda de actividad
- US/Detalle de instructor
- US/Detalle de actividad
- US/Actividades por instructor
- US/Detalle de actividad (instructor)

## EPIC/Gestión de socios

- US/Eliminación de cuenta
- US/Edición de información personal
- US/Baja de actividad
- US/Inscripcion a actividad
- US/Listado de actividades inscriptas

## EPIC/Funcionalidades para administradores

- US/Creacion de usuarios
- US/Inscripcion global
- US/Listado completo de usuarios
- US/Detalle completo de usuario
- US/Eliminación de usuarios
- US/Baja global

## EPIC/Funcionalidades para instructores

- US/Listado de socios
- US/Inscripcion de socio por instructor
- US/Baja de socio por instructor
- US/Registro de socios por instructor
- US/Detalle de socio

## EPIC/Gestión de rutinas

- US/Creación de rutina
- US/Listado completo de rutinas
- US/Detalle de rutina
- US/Formulario de seguimiento de rutina
- US/Gráfico de progreso

## EPIC/Asistente virtual

- US/Integración del backend con Google Gemini AI
- US/Comunicación del frontend con el backend
- US/Procesamiento de mensajes con contexto

## Artefactos

### Sprint planning

#### Sprint 1

Benedetti, Matias:

- US/Registro de socios
- US/Inicio de sesión

#### Sprint 2

Bassi Servant, Tomas:

- US/Detalle de instructor
- US/Detalle de actividad
- US/Actividades por instructor
- US/Detalle de actividad (instructor)
- US/Baja de actividad
- US/Listado de actividades inscriptas

Benedetti, Matias:

- US/Listado de actividades
- US/Búsqueda de actividad
- US/Eliminación de cuenta
- US/Edición de información personal
- US/Inscripción a actividad

#### Sprint 3

Bassi Servant, Tomas:

- Documentación: Medición funcional, especificaciones de requisitos funcionales

Benedetti, Matias:

- US/Creación de usuarios
- US/Inscripción global
- US/Listado completo de usuarios
- US/Detalle completo de usuario
- US/Eliminación de usuarios
- US/Baja global

#### Sprint 4

Bassi Servant, Tomas:

- US/Listado de socios

- US/Inscripcion de socio por instructor
- US/Baja de socio por instructor
- US/Registro de socios por instructor
- US/Detalle de socio

Benedetti, Matias:

- Documentación: Gestión de cambio, decisiones clave, arquitectura del sistema

## Sprint 5

Bassi Servant, Tomas:

- US/Creación de rutina
- US/Listado completo de rutinas
- US/Detalle de rutina
- Documentación: Cierre, comparación estimación vs real, análisis de desviaciones

Benedetti, Matias:

- US/Formulario de seguimiento de rutina
- US/Gráfico de progreso
- US/Integración del backend con Google Gemini AI
- US/Comunicacion del frontend con el backend
- US/Procesamiento de mensajes con contexto

## Sprint review

Al finalizar cada sprint, el equipo de desarrollo presentó el incremento funcional completado al docente. En estas reuniones se mostraron las historias de usuario (US), que se acordaron en el sprint anterior, para validar que el producto cumplía con los requisitos definidos.

## Sprint retrospective

### Sprint 1

¿Qué funcionó bien?

Mantener la división de Épicas y US que se utilizaron en el primer cuatrimestre del segundo año, dedicadas al backend en Spring Boot.

¿Qué se puede mejorar?

Al ya tener realizado el backend, y esta etapa del trabajo se centra en el frontend, se pueden hacer más US en cada sprint.

Acciones de mejora para el próximo sprint

Hacer mayor cantidad de US.

## Sprint 2

¿Qué funcionó bien?

Realizar más US por desarrollador hizo que se vieran mayores avances en el proyecto. No se sintió tanto el esfuerzo.

¿Qué se puede mejorar?

Las prioridades de US y definir bloqueos entre US.

Acciones de mejora para el próximo sprint

Reducir la carga de US al desarrollador que lo necesite, el sprint 3 se ve afectado por parciales y entregas de otras materias.

## Sprint 3

¿Qué funcionó bien?

Hubo una buena división de tareas, donde uno de los desarrolladores se dedicó de lleno al código y el otro comenzó con la documentación.

¿Qué se puede mejorar?

La documentación llevará más tiempo de lo pensado, es necesario dedicarle más tiempo por sprint.

Acciones de mejora para el próximo sprint

Cambiar de roles, un desarrollador codea y el otro documenta.  
Investigar acerca de tecnologías a aplicar en el proyecto.

## Sprint 4

¿Qué funcionó bien?

Se avanzó bastante rápido y se terminó antes de la fecha límite, por lo tanto se decidió agregar más tiempo a documentar.

El tiempo dedicado a investigación, facilitó la toma de decisiones.

¿Qué se puede mejorar?

Definir los ADR y la solicitud de cambio, ya que retrasan otras secciones de la documentación.

Acciones de mejora para el próximo sprint

Hacer foco en secciones que son cruciales para avanzar y poder completar la documentación.

## Sprint 5

### ¿Qué funcionó bien?

Uno de los desarrolladores pidió vacaciones de su trabajo, para poder en este sprint dedicarnos absolutamente al proyecto.

La comunicación fue un factor crucial para poder terminar el proyecto a tiempo.

### ¿Qué se puede mejorar?

Posiblemente los story points de las US, ya que al ganar más experiencia durante este proyecto, la estimación de estos puntos podría variar.

### Acciones de mejora para el próximo sprint

Definir un detalle del paso a paso para los próximos requisitos funcionales requeridos para presentar este proyecto como tesis.

## Gestión de cambios

| SOLICITUD DE CAMBIO #001                                                                                                                                                                                                                                                                                                       |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <u>Fecha de solicitud:</u> 16/10/2025                                                                                                                                                                                                                                                                                          |  |
| <u>Solicitante:</u> Cliente                                                                                                                                                                                                                                                                                                    |  |
| <u>Cambio propuesto:</u> Cambiar la implementación del backend del módulo de rutinas de Java (Spring Boot) a Supabase (Backend as a Service). El frontend se comunicaría directamente con la API de Supabase para todas las operaciones relacionadas con las rutinas, como la creación, asignación y seguimiento del progreso. |  |
| <u>Justificación:</u> Se estima un ahorro significativo de tiempo y costo de desarrollo al utilizar un servicio BaaS en lugar de codificar el backend manualmente. Supabase auto-genera la API REST necesaria y gestiona la base de datos, reduciendo drásticamente la necesidad de escribir código de Java.                   |  |
| <u>Impacto en el proyecto:</u>                                                                                                                                                                                                                                                                                                 |  |
| 1. <u>Alcance:</u> Se verán afectados los RF del RF-29 al RF-36. Ahorro de 1.57 KLOC (Líneas de Código) del backend en Java. Cálculo basado en 27 UFP, FA de 1.06 y factor de conversión de 55 LOC/PF                                                                                                                          |  |
| 2. <u>Tiempo:</u> Ahorro estimado de 4.11 Meses (de tiempo de desarrollo del backend). Cálculo basado en modelo COCOMO: 1.57 KLOC = 3.86 PM y TDEV de 4.11 meses                                                                                                                                                               |  |
| 3. <u>Costo:</u> Ahorro estimado de 3,860 USD (en esfuerzo de desarrollo del backend). Cálculo basado en 3.86 PM y costo de 1000 USD/persona-mes                                                                                                                                                                               |  |
| 4. <u>Recursos:</u> Requiere la implementación de Supabase como Backend as a Service.                                                                                                                                                                                                                                          |  |
| 5. <u>Riesgos:</u> La introducción de Supabase junto al backend de Spring Boot genera un desafío a la hora de sincronizar los datos de los usuarios.                                                                                                                                                                           |  |



Decisión: Rechazada

Justificación de la decisión:

- Requisito de tesis: El desarrollo del backend es un requisito académico. Usar un BaaS anularía la evaluación de esta competencia.
- Familiaridad vs curva de aprendizaje: El equipo de desarrollo ya tiene familiaridad con las tecnologías utilizadas. Introducir Supabase añade una curva de aprendizaje no planificada que genera incertidumbre.

Fecha de decisión: 08/11/2025

Responsable de la decisión: Equipo de desarrollo

## Cierre

### Comparación estimación vs real

**Tabla Comparativa Final**

| Métrica                 | Estimación Inicial (COCOMO) | Valor Real Ejecutado | Desviación Absoluta | % Desviación |
|-------------------------|-----------------------------|----------------------|---------------------|--------------|
| Puntos de Función       | 145.22 PF                   | 145.22 PF            | 0                   | 0%           |
| Esfuerzo (personas-mes) | 21.29                       | 7 (3.5 * 2)          | -14.29              | -67.12%      |
| Tiempo (meses)          | 8.3                         | 3.5                  | -4.8                | -57.83%      |
| Costo total (USD)       | 21248                       | 7000                 | -14.248             | -67.06%      |
| Requisitos planificados | 40                          | 40                   | 0                   | 0%           |

|                               |    |    |   |    |
|-------------------------------|----|----|---|----|
| <b>Requisitos completados</b> | 40 | 40 | 0 | 0% |
|-------------------------------|----|----|---|----|

## Análisis de desviaciones

La alta desviación negativa en **Esfuerzo, Tiempo y Costo** (aprox. -67%) indica que el valor real ejecutado fue significativamente menor que lo predicho por el modelo COCOMO.

### Esfuerzo, Tiempo y Costo (aprox. -67%)

#### Causa Raíz de la Desviación

La causa principal es la **inadecuación del modelo COCOMO Básico** para el contexto del proyecto académico:

- **COCOMO Sobrestima la Gestión:** El modelo está diseñado para proyectos de mayor escala y tradicionales, incluyendo un gran esfuerzo de gestión, documentación formal, pruebas de sistema.
- **Productividad del Equipo:** El equipo demostró una productividad neta alta, completando el trabajo teórico de 21.29 PM en solo 7 PM reales. La motivación del Trabajo Final puede haber sido un factor clave para la eficiencia por encima del estándar del modo Orgánico.

#### ¿Se podría haber previsto?

**Sí.** Se podría haber previsto una desviación tan grande. La lección es que para proyectos pequeños (7.99 KLOC), COCOMO Básico tiende a sobreestimar el esfuerzo, por lo tanto, para este tipo de proyectos lo mejor hubiera sido utilizar otra alternativa de medición.

#### ¿Qué acciones se tomaron al detectarla?

La desviación fue un **resultado favorable**. La acción tomada fue:

- **Mantener el Alcance:** Mantuvimos el 100% del alcance planificado de los 40 RFs.

#### ¿Cómo impactó en el resultado final?

El impacto fue totalmente positivo para el resultado del proyecto:

- El proyecto se completó al 100% del alcance, con una **entrega anticipada** de 4.8 meses respecto a la estimación formal.
- Se validó la alta eficiencia del equipo al lograr un **ahorro teórico del 67%** en el costo estimado.